



ENCS4320 — Applied Cryptography

Design Decisions: Options, Choice, and Justification

Team Members

Student 1name : Besan Maaly ID: 1222776 Section: 2

Student 2name : Shahd Shreteh ID: 1210444 Section: 2

Student 3name : Dania Abuayyash ID: 1210464 Section: 2

Instructor: Dr. Ahmad Shawahna

Date: 12 / 9 / 2026

7.1 Password Protection

Option	Status	Reason
PBKDF2	Rejected	Not memory-hard; its work factor is heavily amortised by GPU/FPGA/ASIC parallelism
bcrypt	Rejected	Improves on PBKDF2, but its memory footprint is effectively fixed at ~4 KB and cannot be raised to keep pace with modern hardware
scrypt	Rejected	Genuinely memory-hard, but its three tunable parameters (N, r, p) are less standardised and harder to justify precisely than Argon2's
Argon2id	Selected	Winner of the Password Hashing Competition (2015); combines memory-hardness with resistance to side-channel leakage through its hybrid access pattern
A custom construction on a plain hash function, without memory expansion	Rejected	Trivially fast on a GPU; fails every requirement of Section 5.1 outright

7.2 Protecting Document Contents

Option	Status	Reason
AES-GCM (ready-made AEAD mode)	Rejected	An excellent AEAD construction in principle, but a correct, constant-time hand implementation of GHASH (multiplication over $GF(2^{128})$) is high risk and disproportionately difficult to verify within the projects 70% from scratch requirement
AES-CCM	Rejected	Requires two passes of CBC-MAC-style computation implemented by hand, with no streaming support and no meaningful advantage over the chosen alternative for this project's scope
Stream cipher + Poly1305 / one-time MAC	Rejected	Attractive in theory, but a correct implementation of Poly1305 arithmetic ($\text{mod } 2^{130} - 5$) is at least as error prone as GHASH and does not reduce risk relative to the chosen option
AES-256-CBC + HMAC-SHA256 (hand-built Encrypt-then-MAC)	Selected	Both AES (Rijndael) and HMAC are fully and clearly specified (FIPS-197 and RFC 2104 respectively); implementing each separately and composing them ourselves satisfies the 70% from-scratch requirement without the parallel-field-arithmetic risk of GCM

Decision and Composition Order

Encrypt-then-MAC: the plaintext is encrypted first under AES-256-CBC, and HMAC-SHA256 is then computed over the resulting IV, ciphertext, and any bound metadata; the resulting tag is appended to the object.

- This ordering — rather than MAC-then-Encrypt or Encrypt-and-MAC — allows the tag to be verified before any decryption is attempted: data that has not been authenticated is never passed to the AES-CBC decryption routine, which closes off padding-oracle attacks at the root, since the only observable failure is a single, undifferentiated “verification failed” outcome.
- Encrypt-and-MAC (computing the MAC directly over the plaintext) is rejected because, in some constructions, the tag itself can leak information about the plaintext. MAC-then-Encrypt is rejected because it forces full decryption before any tampering is detected — wasting computation and reopening the door to the classic padding-oracle attacks this design is meant to avoid.

7.3 Public-Key Cryptography

Option	Status	Reason
RSA (key transport and signatures)	Rejected	Requires large keys (≥ 3072 bits) to reach a security level comparable to AES-128/256, big integer operations are correspondingly expensive, and a correct hand implementation of the required padding schemes (OAEP/PSS) is high risk
Finite-field Diffie–Hellman (classical DH)	Rejected	Needs group parameters of roughly the same bit length class as RSA to reach an equivalent security level, and is slower than elliptic-curve operations at that level
NIST P-256 with ECDSA	Rejected	A well documented, standard Weierstrass curve, but its arithmetic requires handling special cases (the point at infinity, multiple addition formulas) that are easy to implement incorrectly, and ECDSA signatures are highly sensitive to nonce quality a repeated nonce leaks the private key immediately
Curve25519 (X25519) for agreement, Ed25519 for signatures	Selected	A Montgomery-form curve with complete addition formulas and no special cases, explicitly designed to minimise implementation error, Ed25519 derives its nonce deterministically from the message and key, eliminating the single most common real world cause of ECDSA failure

Why This Curve Specifically

Its algorithmic description (the Montgomery ladder) is unambiguous, is constant-time by construction without extra engineering effort, and does not demand the same degree of point-validation rigour that Weierstrass curves require directly reducing the team's exposure to implementation error.

Quantified Comparison Against the Rejected Alternative (RSA)

To reach roughly a 128-bit security level (comparable, in brute-force search difficulty, to a 256-bit symmetric key, after accounting for the square-root speed-up that Pollard's rho gives against elliptic-curve discrete logarithms): a 256-bit Curve25519 key suffices, whereas RSA requires a modulus of roughly 3072 bits for the same level about twelve times larger. A single elliptic-curve operation is correspondingly tens of times cheaper computationally than an equivalent RSA operation, because RSA's cost grows roughly cubically with key length while elliptic-curve cost grows roughly linearly with curve size.

7.4 Establishing Trust in a Public Key

Option	Status	Reason
Mandatory out-of-band fingerprint comparison for every new contact	Rejected	Offers the strongest guarantee, but assumes a second, already-trusted channel between every pair of users before their first interaction unrealistic for a multi-user system where users, like Layla and Omar, have never previously met
A small Certificate Authority (CA) implemented by the team	Rejected	Reintroduces a centrally trusted third party, and requires certificate issuance/revocation infrastructure.
Trust-On-First-Use (TOFU) with subsequent change detection	Selected	Requires no additional infrastructure; mirrors models that are proven in practice (SSH, Signal, simplified PGP web-of-trust), reframes the problem from "prevent every attack from day one" to "detect any later change and alert the user".

Decision and Its Assumptions

On first contact between two users, the client accepts the counterpart's public key exactly as delivered by the server and stores it locally (Trust-On-First-Use). On every subsequent contact, the client compares the key received against the one stored locally: any discrepancy is surfaced to the user as an explicit, non-dismissible warning, rather than being silently accepted.

Stated assumption: the very first contact between two users who have never previously communicated is not, at that precise moment, under an active man-in-the-middle attack. This is a known and accepted assumption of TOFU-based models.

7.5 Digital Signatures Versus MACs

	MAC (HMAC-SHA256)	Digital Signature (Ed25519)
Where used	Integrity of the ciphertext while stored or in transit (Section 7.2)	Owner's signature over the content hash and metadata at upload time
Who holds the key	A symmetric key, known to at least two parties (sender and server/recipient after unwrapping)	A private key known exclusively to the document's owner
Provides non-repudiation?	No: any party holding the shared key could have produced the same tag, so authorship cannot be proven	Yes: verification succeeds only against the public key corresponding to a single private key, held solely by the document's owner

7.6 Architecture and Message Formats

Option	Status	Reason
A shared directory simulating the network (file-based)	Rejected	This option makes it difficult to convincingly demonstrate real-time behaviours (automatic rejection, immediate feedback) during a live presentation
Peer-to-peer with the server acting only as a directory	Rejected	Adds networking complexity (NAT traversal, direct connections) without any additional security benefit under the given threat model
Client-server over TCP sockets	Selected	Matches the scenario in Section 2 directly (sign-up, login, upload, share, retrieve) with the least structural complexity, and makes the live rejection scenarios required by Section 8 straightforward to demonstrate

Summary of Decisions

Decision	Choice
7.1 Passwords	Argon2id (memory-hard, hybrid access pattern)
7.2 Document encryption	AES-256-CBC + HMAC-SHA256, Encrypt-then-MAC
7.3 Public-key crypto	X25519 (agreement) + Ed25519 (signatures)
7.4 Key trust	Trust-On-First-Use with change detection, plus an optional verification code
7.5 MAC vs. signature	MAC for transport integrity; signature for non-repudiation
7.6 Architecture	Client-server over TCP sockets, fixed/length-prefixed message format